

Testing

The rule

Coverage is not a goal. Tests exist only for the few things that are genuinely load-bearing and easy to get subtly wrong. Everything else is validated by exploratory verification against the module's Definition of Done. Hand-written exhaustive CRUD tests are explicitly not wanted — they cost more than they protect.

Every test file's header states **which rubric bullet or gotcha it protects against**, so the output of **make check** doubles as a rubric checklist.

Running

```
make check      # lint + types + tests, both apps – the only gate
make test-api   # backend only
make test-web    # frontend only
```

Everything runs inside containers. Integration tests create and drop a dedicated **finmanager_test** database on the same Postgres container; they never touch the development data.

What is covered today

Backend — **apps/api/tests/**

File	Protects
unit/test_money_decimal.py	«no float money anywhere» — floats are rejected at the boundary, rounding is half-up to cents, repeated addition stays exact
unit/test_lego_roi.py	M9 unrealized ROI math: positive, negative, gift (cost 0 → None), no value set (→ None) , and both at once
unit/test_lego_storage_capacity.py	M9 storage capacity math and the rule that capacity_pct is a human estimate, never derived from stored_count
integration/test_lego_lifecycle.py	Ownership transitions, KPI exclusion of SOLD/GIFTED , sale fields cleared on return, delete guards (model with live copies, location with assigned copies), find-or-create by set number, value_updated_at stamping, and that every mutation writes an AuditLog row
integration/test_household_rbac.py	VIEWER is read-only, OWNER gate, «todas» refuses to guess an owner, read-only entities refuse writes,

File	Protects
	and the last-OWNER safeguard
<code>integration/test_document_security.py</code>	Magic-byte validation beats the supplied filename, disguised executables are refused, files land outside any web root, deduplication by hash, and signed URLs that fail closed on tamper or expiry

Frontend — co-located `*.test.ts(x)`

File	Protects
<code>src/lib/format.test.ts</code>	pt-PT EUR formatting through the single money util; em dash (never <code>0 %</code>) for a null ROI; pt-PT date and thousands separators
<code>src/features/lego/constants.test.ts</code>	M9 FR-9.10 — marketplace links are built from a <code>set_number</code> template, nothing stored or fetched

Conventions

- `tests/unit/` holds pure domain logic with **no I/O**. If a test needs a database it belongs in `tests/integration/` and carries `pytest.mark.integration`.
- One file per concern, named after what it protects (`test_lego_roi.py`), not after the module it happens to import.
- Fixtures live in `tests/conftest.py`: `db`, `household`, `owner`, `entity` and an `api_client` that overrides the request-scoped session.
- Frontend tests sit next to the code they cover; end-to-end coverage is reserved for the crown-jewel flows once those modules exist (receipt review, review-queue confirm).

Adding a test

Ask: *if this were silently wrong, would money, history or a lifecycle guard be wrong?* If not, do not write the test. If yes, write it, and start the file with a docstring naming the rubric bullet it defends.